



Université de
Sherbrooke

Université de Sherbrooke

SQL

Types élémentaires et prédéfinis

SQL_01

Christina KHNAISSER (christina.khnaisser@usherbrooke.ca)

Luc LAVOIE (luc.lavoie@usherbrooke.ca)

(les auteurs sont cités en ordre alphabétique nominal)

Scriptorum/Scriptorum/SQL_01-Types-elementaires, version 1.0.0.a, en date du 2026-01-11

— document de travail, ne pas citer —

Plan

Introduction	3
1. Types prédéfinis usuels.	4
2. Fonctions et opérateurs prédéfinis	19
Conclusion	27
Références.	29
Définitions	30

Introduction

Le présent document a pour but de présenter une synthèse des types prédéfinies usuels qu'on retrouve dans la norme SQL.

1. Types prédéfinis usuels

1.1. Types prédéfinis de SQL

Les principaux types prescrits par la norme ISO 9075:2023.

1.1.1. Type booléen

BOOLEAN

{ FALSE | TRUE | UNKNOWN }

1.1.2. Types textuels

- CHARACTER (CHAR) : texte de longueur fixée et constante
- CHARACTER VARYING (VARCHAR) : texte de longueur variable

Dans les deux cas, il faut préciser le nombre de caractères :

- exacte dans le cas de CHAR
- maximale dans le cas de VARCHAR

Exemple 1. Exemples de types textuels

'IFT187 '::CHAR(12) -- exactement 12 caractères

'IFT187 '::VARCHAR(12) -- au plus 12 caractères

1.1.3. Types numériques

Il est important de choisir le bon type et de restreindre le domaine de valeur au plus juste.

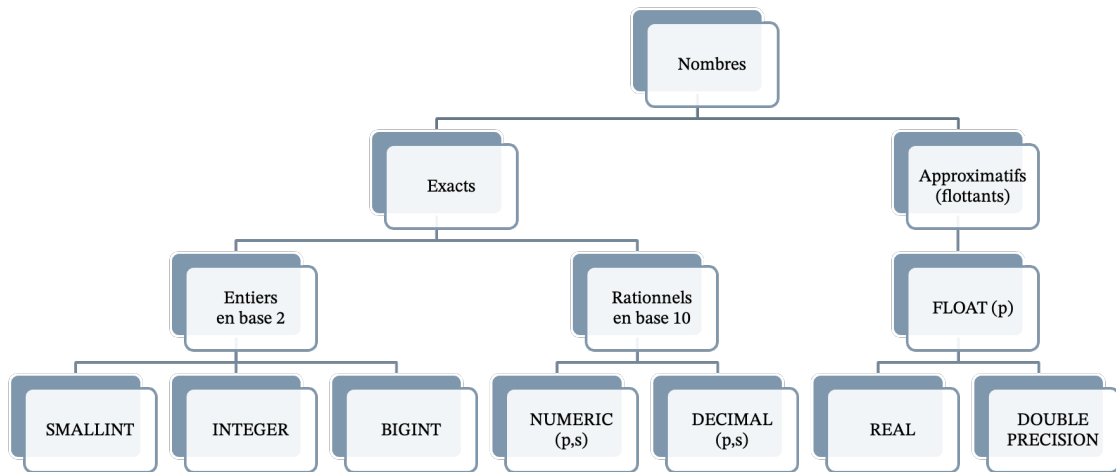


Figure 1. Types prédéfinis numériques ISO

Entiers

Les nombres entiers (sans décimale) sont bornés de différentes étendues. Les bornes devant être définies par chaque dialecte tout en respectant la contrainte : $\{-32767, \dots, 32767\} \subseteq \text{SMALLINT} \subseteq \text{INTEGER} \subseteq \text{BIGINT}$

Rationnels

Les nombres exacts à base 10 avec :

- `p(precision)` : un entier positif indiquant le nombre **total** de chiffres significatifs. Par défaut, la valeur est égale à 0. La valeur maximale est à 1000.
- `s(scale)` : un entier (positif ou négatif) indiquant nombre de chiffres décimaux de la partie fractionnaire. Par défaut, la valeur est égale à 0. La valeur maximale est celle de la précision.

Depuis 2016, `NUMERIC` et `DECIMAL` sont équivalents.
Il est recommandé d'utiliser `NUMERIC`.

Exemple 2. Exemples de types rationnels

`NUMERIC(3,1)` -- permet d'enregistrer des valeurs entre -99.9 et 99.9

`NUMERIC(2, -3)` — permet d'enregistrer des valeurs entre -99000 et 99000 Les valeurs sont alors arrondies à partir de la gauche du point décimal.

`NUMERIC(3, 5)` — permet d'enregistrer des valeurs comprises entre -0,00999 et 0,00999.

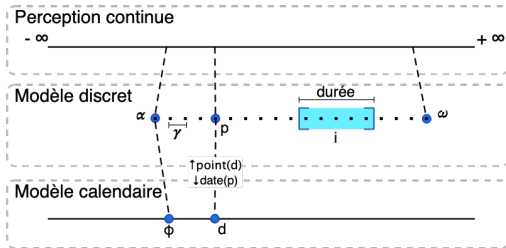
`23.5141::NUMERIC(6,4)` -- exactement 6 chiffres dont 4 chiffres à la partie fractionnaire.

Approximatifs

Les nombres inexacts, SQL prescrit le respect du standard IEEE Standard 754 (Binary Floating-Point Arithmetic) REAL et DOUBLE PRECISION ne sont que des cas particuliers de la notation générale FLOAT (p) où p représente le nombre minimal de bit de la mantisse :

- REAL == FLAOT(1) à FLOAT (24)
- DOUBLE PRECISION == FLAOT(25) à FLOAT (23)

1.1.4. Types temporels



α : point initial (minimum)

ω : point final (maximum)

γ : distance entre deux points consécutifs = $1 / \Gamma$

Γ : nombre de points dans une seconde

i : intervalle quelconque

p, p_a, p_b : points quelconques

d, d_a, d_b : dates quelconques

$\text{durée}(i) = (\text{card}(i)-1) * \gamma$

$\text{date}(\alpha) = \phi$

$\text{point}(\phi) = \alpha$

$p_a < p_b \Rightarrow \text{date}(p_a) \leq \text{date}(p_b)$, mais $\text{point}(\text{date}(p)) = p$ n'est pas garanti

$d_a < d_b \Rightarrow \text{point}(d_a) \leq \text{point}(d_b)$, mais $\text{date}(\text{point}(d)) = d$ n'est pas garanti

Figure 2. Types prédéfinis temporels

- **TIMESTAMP** : un point sur l'axe du temp;

- INTERVAL : une durée
- DATE : une référence à un jour d'un calendrier
- TIME : une référence à un moment de la journée

1.2. Types prédéfinis de PostgreSQL

1.2.1. Type booléen

boolean * TRUE | 't' | 'true' | 'y' | 'yes' | 'on' | '1' * FALSE | 'f' | 'false' | 'n' | 'no' | 'off' | '0'

1.2.2. Types textuels

Tableau 1. Types textuels (PostgreSQL)

Nom	Synonyme	Définition
character(n)	char(n)	longueur fixe, complété par des espaces
character varying(n)	varchar(n)	longueur variable avec limite
text		longueur variable non bornée a priori

1.2.3. Types numériques

Tableau 2. Types numériques (PostgreSQL)

Nom	Taille	Synonyme	Définition
smallint	2	entier de faible étendue	de -32768 à +32767
integer	4	entier habituel	de -2147483648 à +2147483647
bigint	8	grand entier	de -9223372036854775808 à +9223372036854775807
decimal	v	précision indiquée par l'utilisateur, valeur exacte jusqu'à 131072 chiffres avant « . » jusqu'à 16383 chiffres après « . »	numeric
v	précision indiquée par l'utilisateur, valeur exacte	jusqu'à 131072 chiffres avant « . » jusqu'à 16383 chiffres après « . »	real
4	précision variable, valeur inexacte	précision de 6 décimales	double precision

Nom	Taille	Synonyme	Définition
8	précision variable, valeur inexacte	précision de 15 décimales	smallserial
2	entier à incrémentation automatique	de 1 à 32767	serial
4	entier à incrémentation automatique	de 1 à 2147483647	bigserial

1.2.4. Types temporels

Tableau 3. Types temporels (PostgreSQL)

Nom	Taille	Description	Min	Max	Résolution
timestamp [(p)] [without time zone]	8	date et heure sans fuseau horaire	4713 BC	294 276 AD	1 microseconde / 14 chiffres
timestamp [(p)]	with time zone	8	date et heure avec fuseau horaire	4713 BC	294 276 AD
1 microseconde / 14 chiffres	date	4	date seule (pas d'heure)	4713 BC	5 874 897 AD
1 jour	time [(p)] [without time zone]	8	heure seule sans fuseau horaire	00:00:00	24:00:00
1 microseconde / 14 chiffres	time [(p)] with time zone	12	heure seule avec fuseau horaire	00:00:00 +1559	24:00:00-1559 1 microseconde / 14 chiffres

1.2.5. Autres types

- Types binaires
- Types monétaires
- Types géométriques
- Types de recherche texte
- Types UUID
- Types XML
- Types JSON
- Types Tableaux

2. Fonctions et opérateurs prédéfinis

2.1. Fonctions et opérateurs logiques

OR	true	false	unknown
true	true	true	true
false	true	false	unknown
unknown	true	unknown	unknown
AND	true	false	unknown
true	true	false	unknown
false	false	false	false
unknown	unknown	false	unknown
P	NOT P		
true	false		
false	true		
unknown	unknown		
IS	true	false	unknown
true	true	false	false
false	false	true	false
unknown	false	false	true

2.2. Fonctions et opérateurs numériques

Voici une liste (très) partielles :

- Arithmétique sur les entiers :
 - $+$; $-$; $*$; $/$; $\%$
- Arithmétique sur les rationnels et les flottants :
 - $+$; $-$; $*$; $/$; $\%$
 - $\text{abs}(x)$, $\text{round}(x)$
- Fonctions trigonométriques
 - $\sin(x)$, $\cos(x)$, $\tan(x)$

2.3. Fonctions et opérateurs textuels

Voici une liste (très) partielles :

- `char_length (s)`
- `octet_length (s)`
- `s1 || s2`
- `trim ( [[leading | trailing | both] [c] from] s )`
- `position (s, t)`
- `upper (s)`
- `lower (s)`
- `convert (s using x)`
- `translate (s using x)`

Classe	Description	Exemples de valeur en ASCII 7 bits
[:alnum:]	Lettres et chiffres (alnum+digit)	A-Za-z0-9
[:alpha:]	Caractères alphabétiques	A-Za-z
[:blank:]	Espaces et tabulation	\t
[:cntrl:]	Caractères de contrôle	\x00-\x1F\x7F
[:digit:]	Chiffres décimaux	0-9
[:graph:]	Caractères visibles	\x21-\x7E
[:lower:]	Lettres en bas de casse	a-z
[:print:]	Caractères imprimables	\x20-\x7E
[:punct:]	Caractères de ponctuation	[!'"#\$%&'()*+,-./:;<=>?@\^_`{ }~]
[:space:]	Caractères d'espacement	\t \r \n \v \f
[:upper:]	Lettres en haut de casse	A-Z
[:xdigit:]	Chiffres hexadécimaux	A-Fa-f0-9

Figure 3. Classes de caractères Unicode (PostgreSQL)

- Opérateur de conformité **SIMILAR TO**

La fonction booléenne **SIMILAR TO** renvoie **TRUE** si et seulement si le texte de gauche est conforme à l'expression rationnelle représentée par le texte de droite :
texte-gauche **SIMILAR TO** texte-droit

- Opérateur d'extraction **SUBSTR**

substr (s [from d] [for l])

2.4. Fonctions et opérateurs temporels

- `current_timestamp`
- `current_date`
- `current_time`
- `make_timestamp`
- `make_date`
- `make_time`
- `extract(field from x)` `field` := YEAR | MONTH | DAY | HOUR | MINUTE | SECOND pour `x` une valeur de type timestamp, date, time ou interval
- `age(ts0, ts1)` -- différence entre `ts0` et `ts1`
- `age(ts)` -- équivalent à `age(current_timestamp, ts)`

Conclusion

- Le bon usage des types est fondamental au développement de modèles fiables et évolutifs.
- À cet égard, le langage SQL est probablement un des plus riches parmi les langages couramment utilisés.
- Par contre, la variabilité et l'incomplétude de la plupart des dialectes à cet égard rend difficilement transportable (et donc généralisable) l'utilisation de certains des mécanismes de typage. <<<
- On invoque souvent la taille et l'incohérence de la norme ainsi que les problèmes de performance que pourrait entraîner l'adhésion à certaines exigences (comme si un résultat rapide, mais faux était préférable).
- Il y a certes matière à réduire et épurer le langage, voire à en définir un nouveau. En attendant que cela soit fait, il serait avantageux pour tous (développeurs,

informaticiens et maitres d'ouvrage) que les fournisseurs de SGBD adhèrent strictement à la norme plutôt que de pousser des dialectes.

Références

[Date2012a]

Chris J. DATE;

Database Design & Relational Theory;

O'Reilly Media, 2012;

ISBN 978-1-449-33801-6.

PG

[fr] <https://docs.postgresql.fr/18/index.html> (2026-01-05)

[en] <https://www.postgresql.org/docs/18/index.html> (2026-01-05)

Définitions

SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.

[Normes applicables : ISO 9075:2016, ISO 9075:2023]

Produit le 2026-01-20 18:22:15 UTC



Université de Sherbrooke